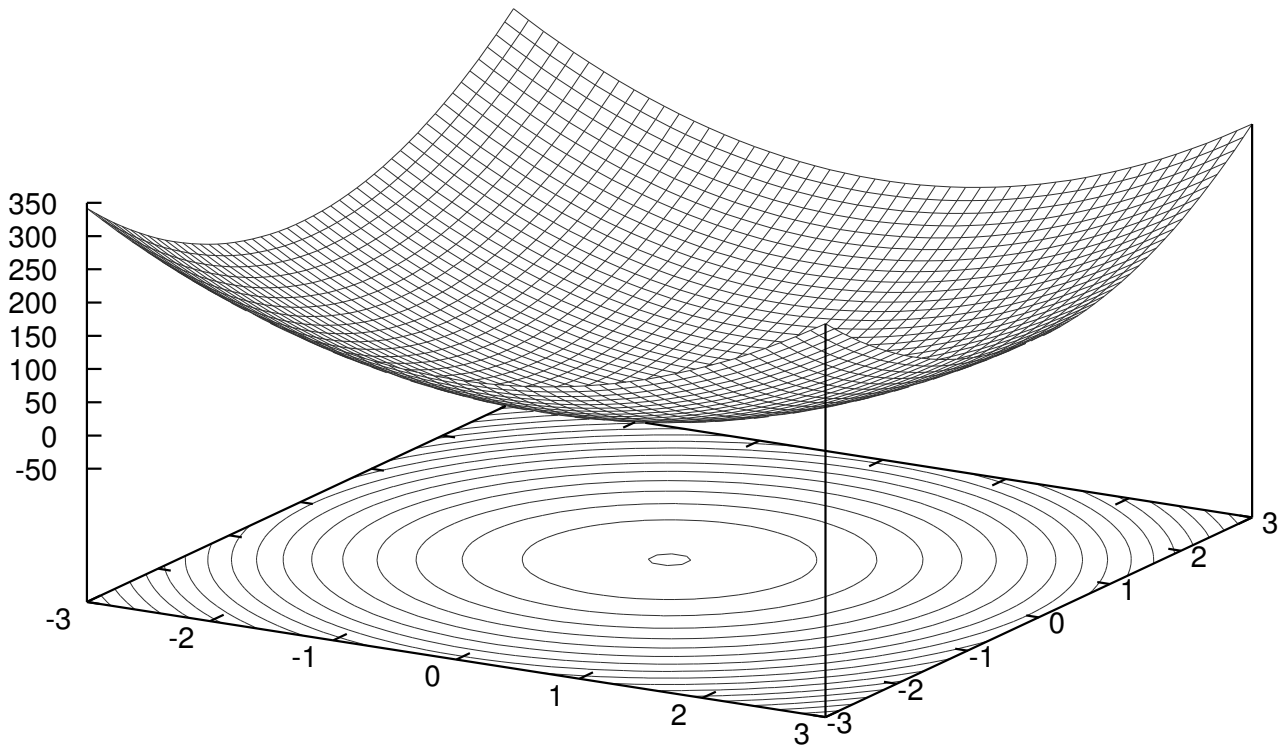
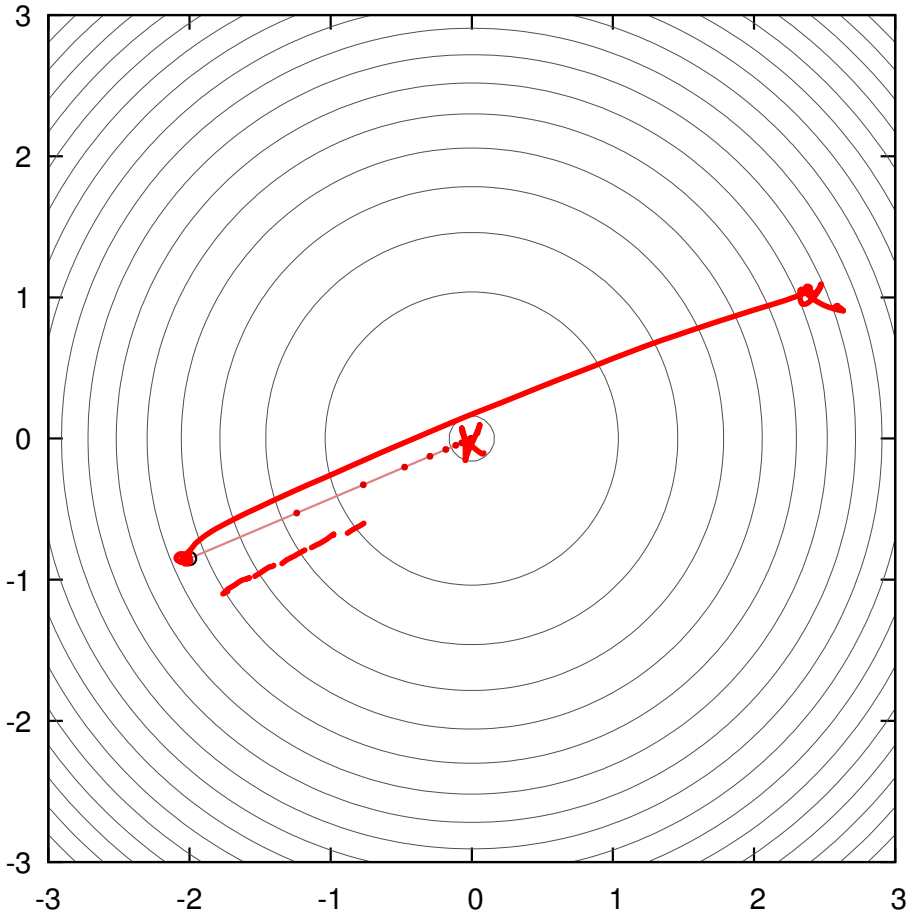
The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.



The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.

$\mathcal{L}$ $\eta = 1.0e - 2$



θ_2

θ_1

$\nabla_\theta \mathcal{L}$

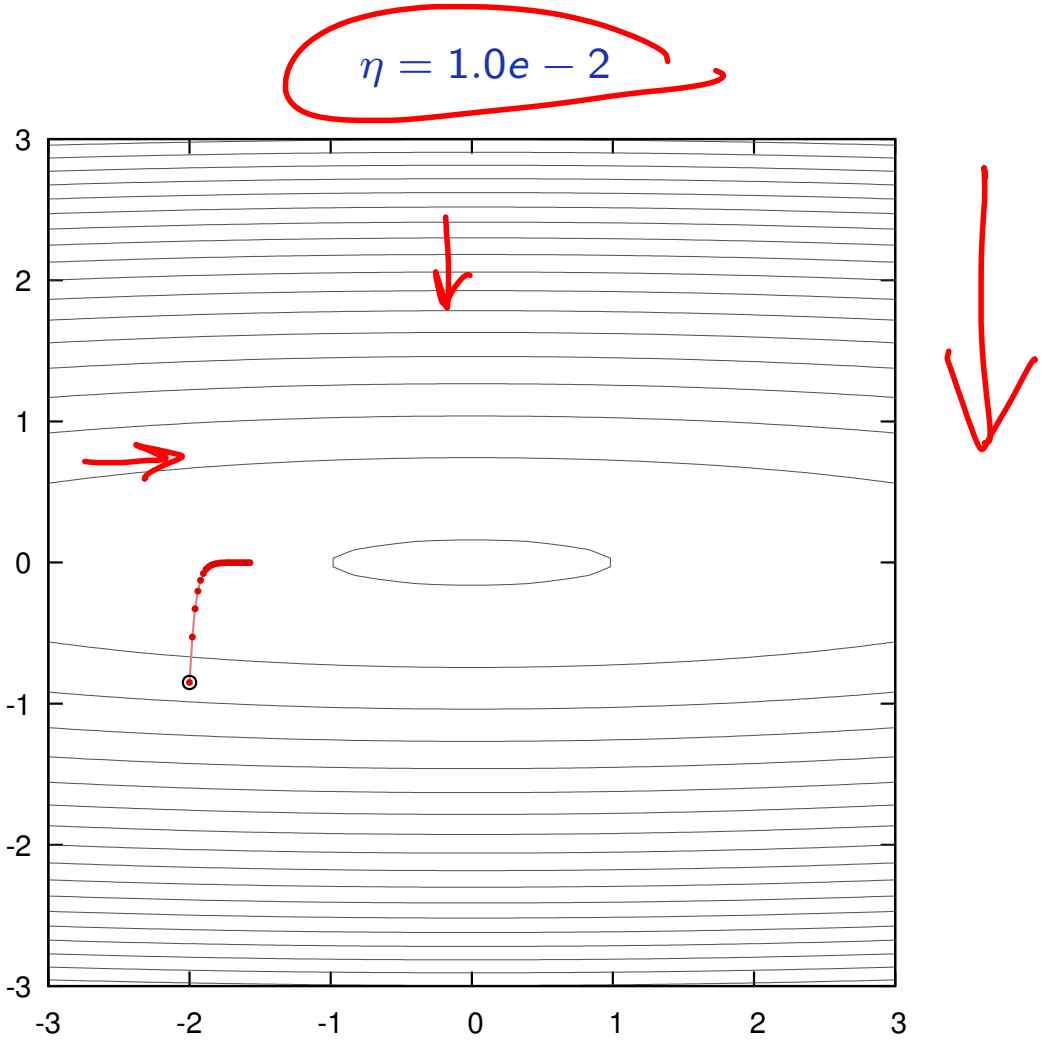
$\nabla^2 \mathcal{L}$

e.values

$\frac{\lambda_{max}}{\lambda_{min}} \approx 1$

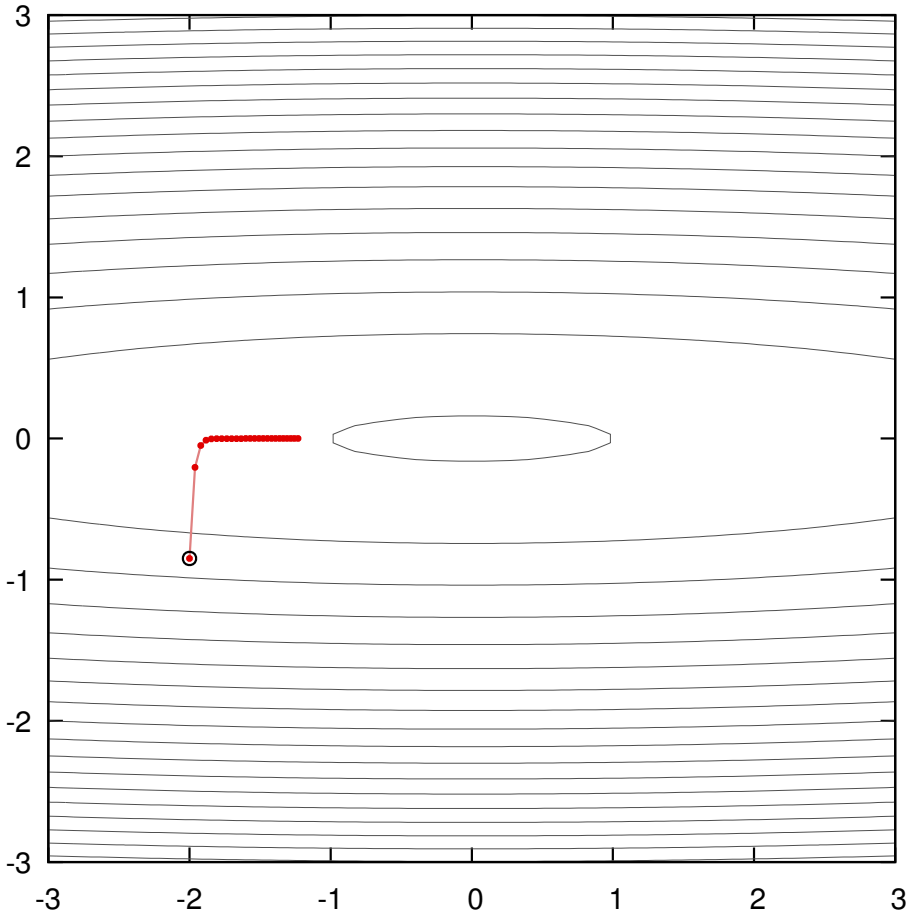
well-conditioned

The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.



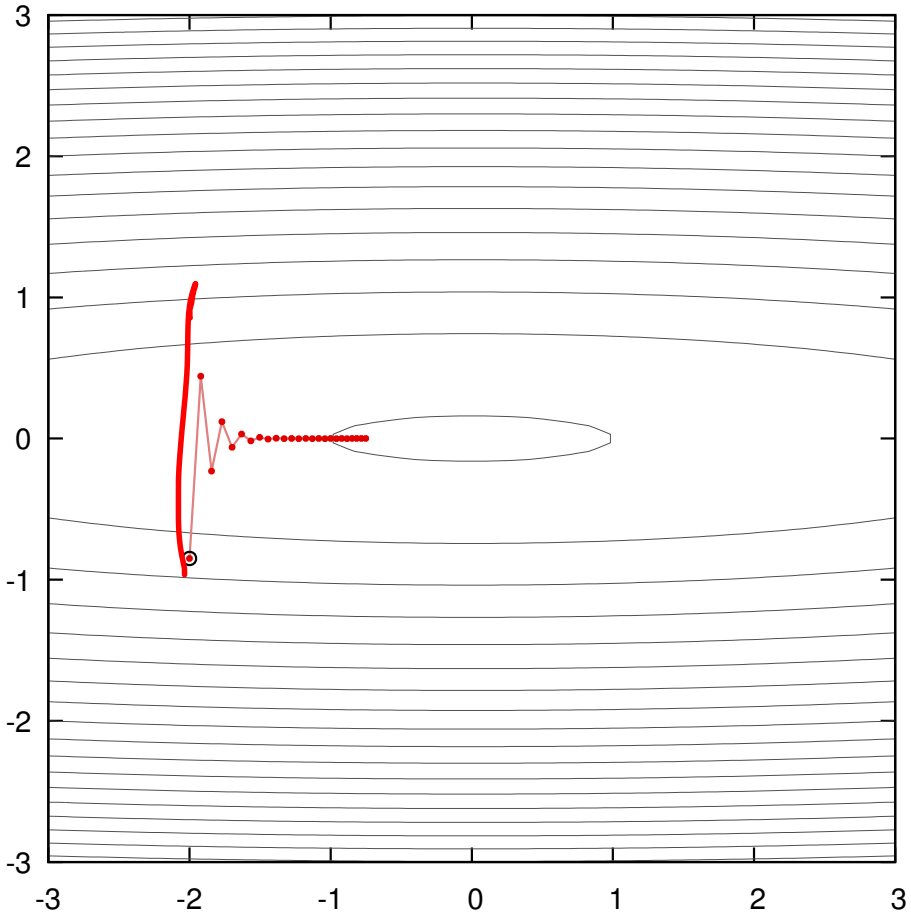
The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.

$\eta = 2.0e - 2$



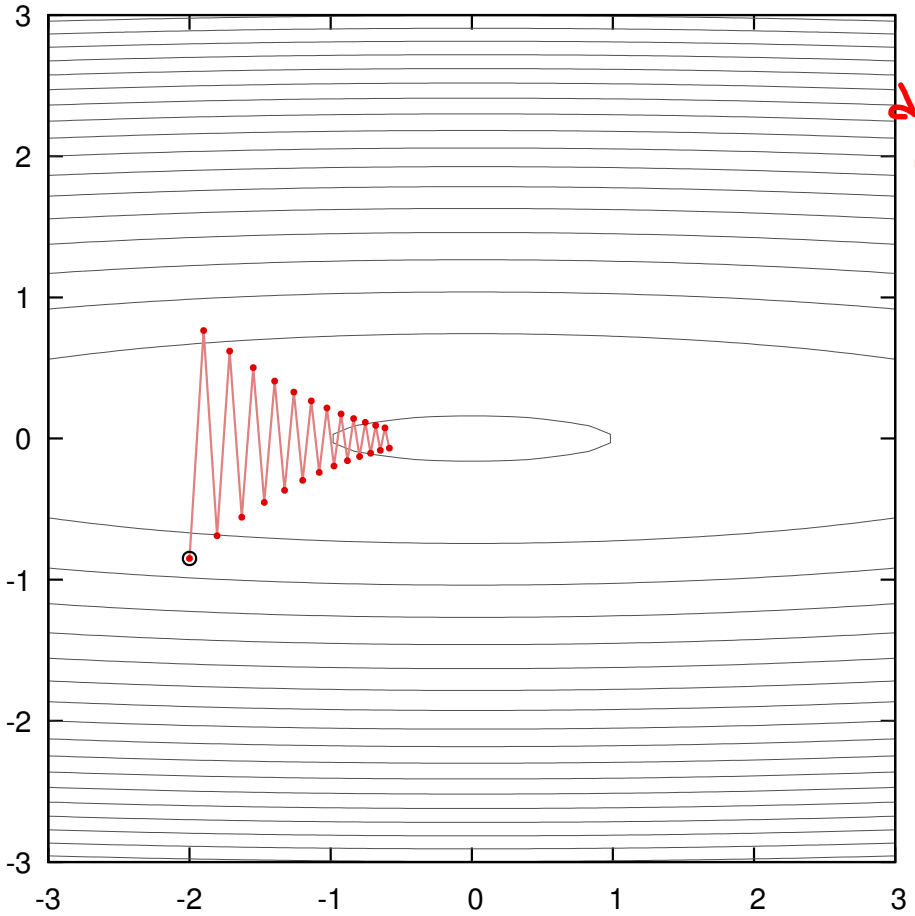
The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.

$\eta = 4.0e - 2$



The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.

$\eta = 5.0e - 2$



$\nabla^2_\theta \mathcal{L}$

λ_{max}

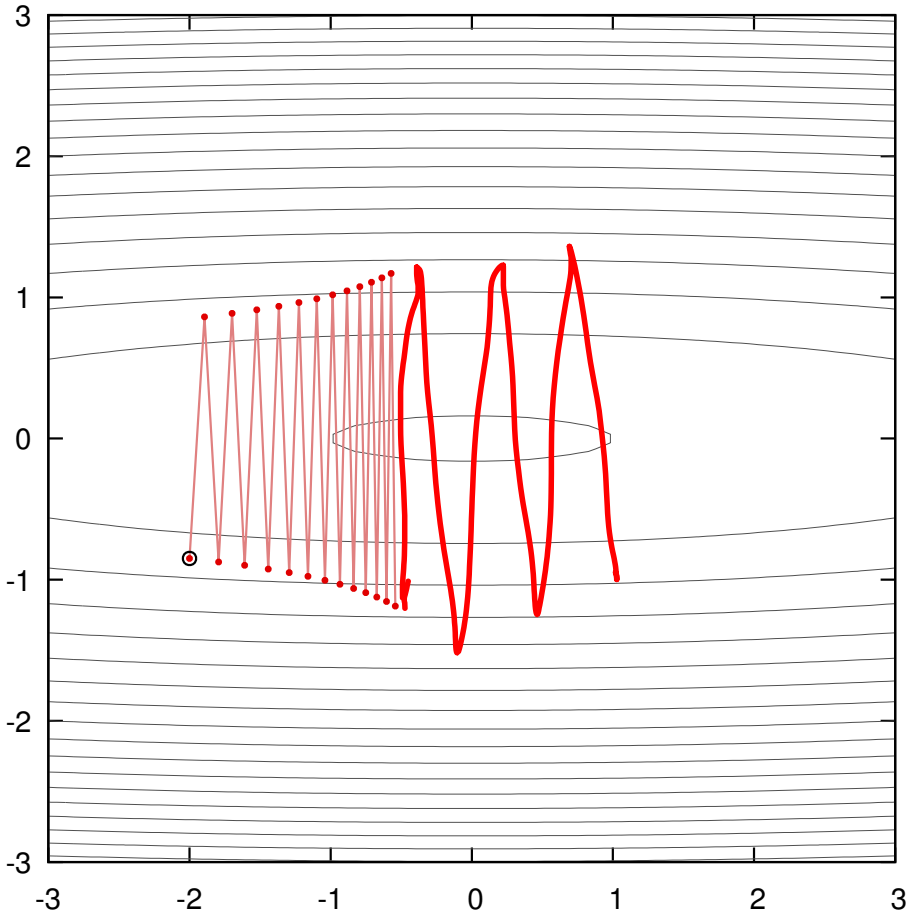
λ_{min}

λ_{max}

λ_{min}

The gradient descent method makes a strong assumption about the magnitude of the "local curvature" to fix the step size, and about its isotropy so that the same step size makes sense in all directions.

$\eta = 5.3e - 2$



The first improvement is the use of a "momentum" to add inertia in the choice of the step direction

$$u_t = \gamma u_{t-1} + \eta g_t$$
$$w_{t+1} = w_t - u_t.$$

(Rumelhart et al., 1986)

With $\gamma = 0$, this is the same as vanilla SGD.

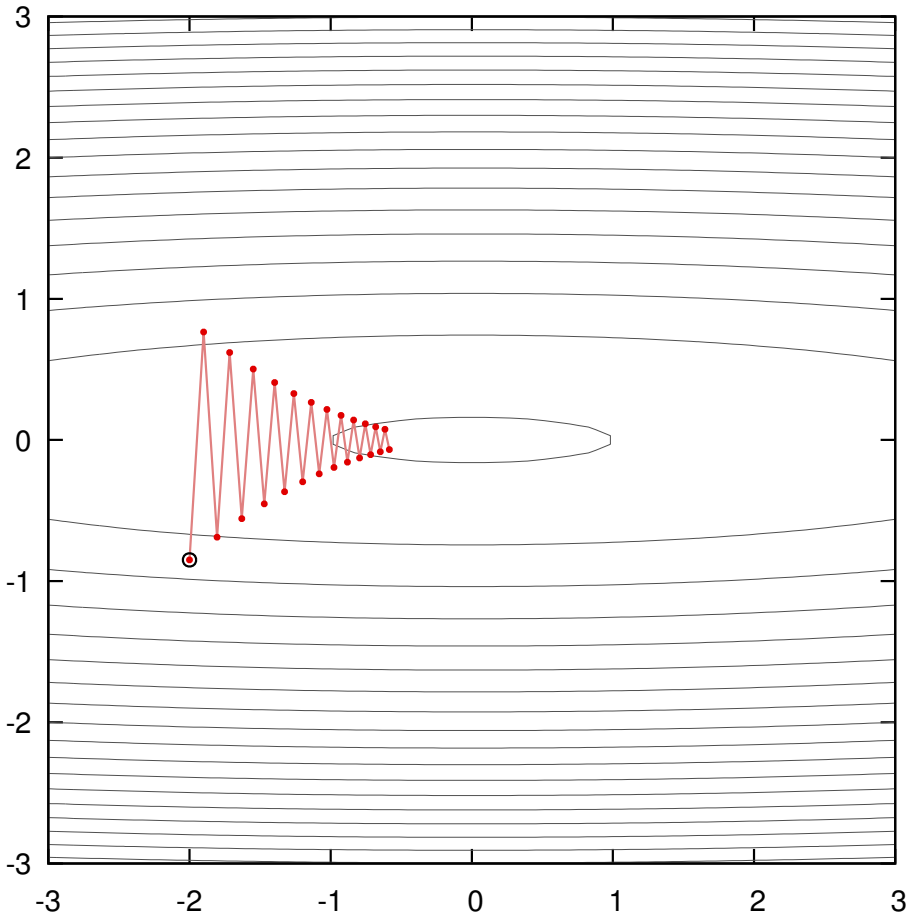
With $\gamma > 0$, this update has three nice properties:

- it can "go through" local barriers,
- it accelerates if the gradient does not change much:

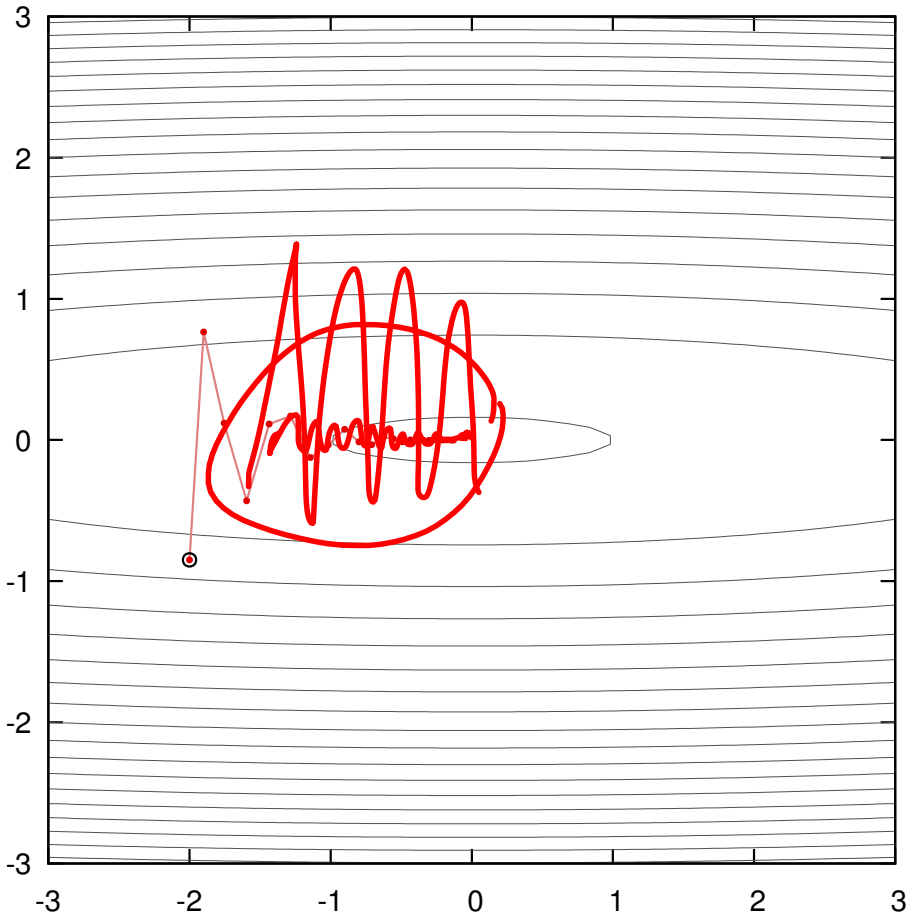
$$(u = \gamma u + \eta g) \Rightarrow \left(u = \frac{\eta}{1-\gamma} g\right),$$

- it dampens oscillations in narrow valleys.

$\eta = 5.0e - 2, \gamma = 0$



$\eta = 5.0e - 2, \gamma = 0.5$



Another class of methods exploits the statistics over the previous steps to compensate for the anisotropy of the mapping.

The Adam algorithm uses moving averages of each coordinate and its square to rescale each coordinate separately.

The update rule is, **on each coordinate separately**

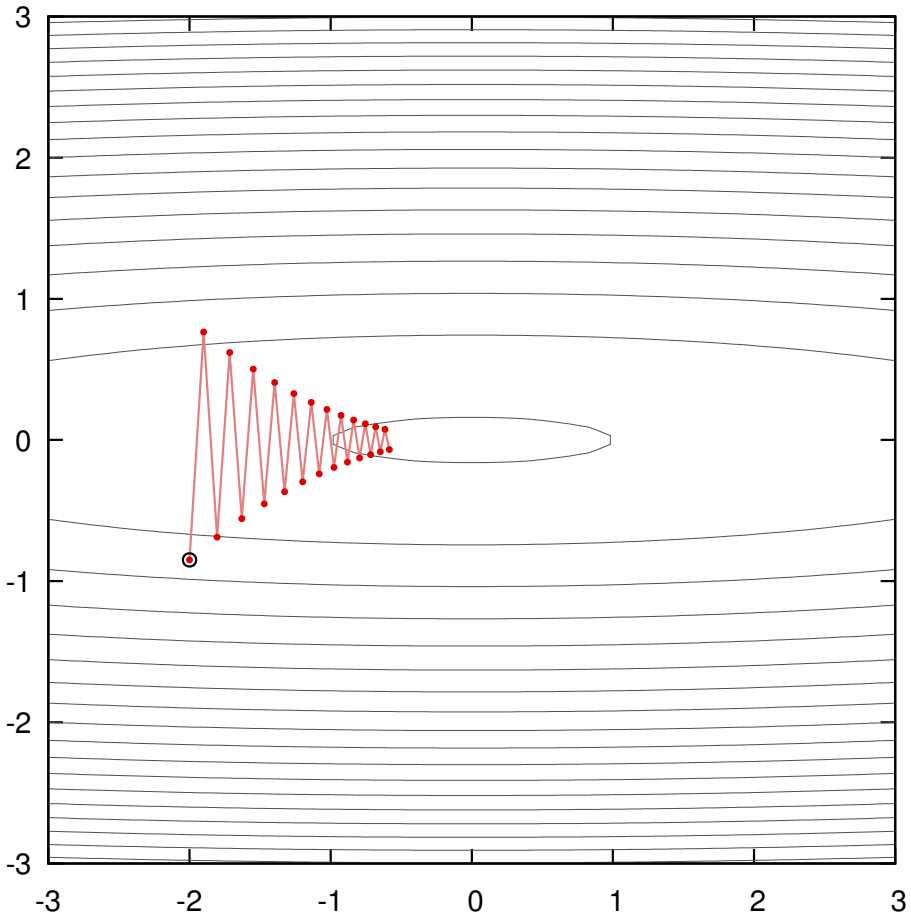
$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
\hat{m}_t &= \frac{m_t}{1-\beta_1} \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{v}_t &= \frac{v_t}{1-\beta_2} \\
w_{t+1} &= w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t
\end{aligned}
$$

(Kingma and Ba, 2014)

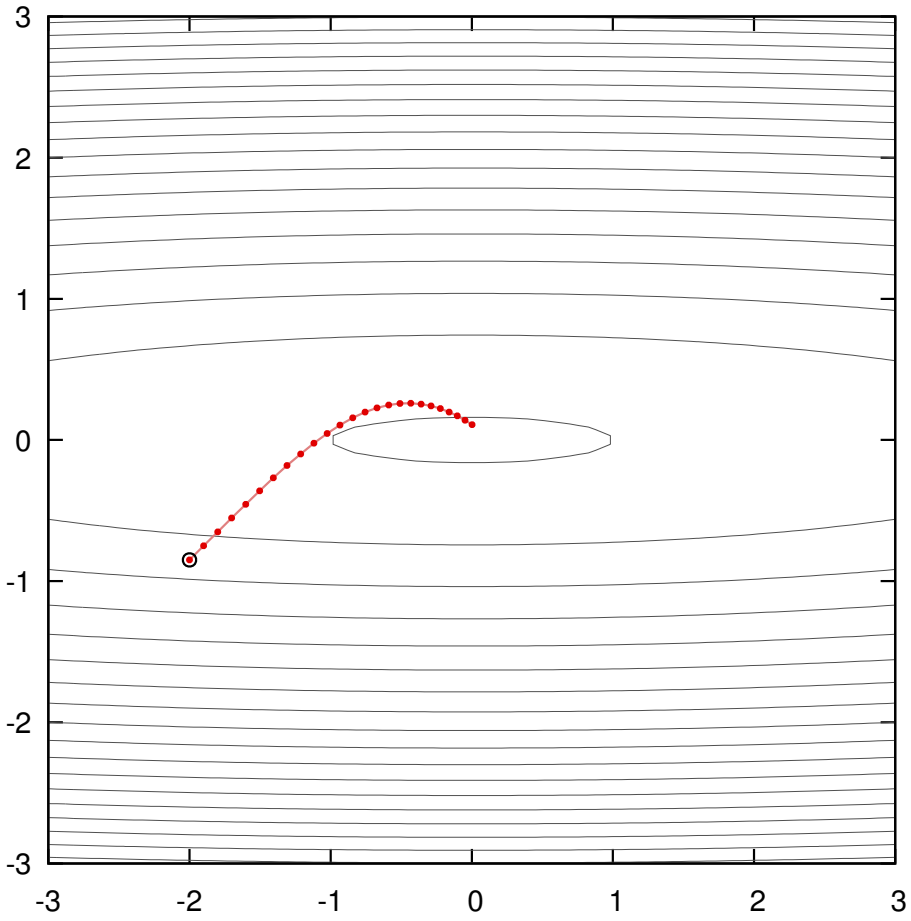
This can be seen as a combination of momentum, with $\hat{m}_t$, and a per-coordinate re-scaling with $\hat{v}_t$.

"vanilla"

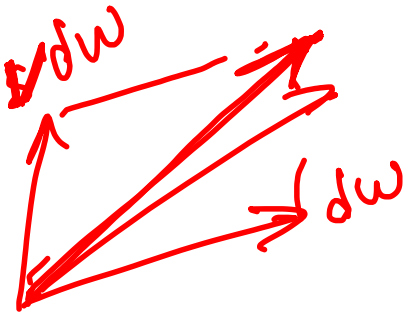
$\eta = 5.0e - 2$



Adam, $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e-8, \eta = 1.0e-1$



momentum



CS231n

These two core strategies have been used in multiple incarnations:

- Nesterov's accelerated gradient,
- Adagrad,
- Adadelta,
- RMSprop,
- AdaMax,
- Nadam ...

- `torch.optim.SGD` (momentum, and Nesterov's algorithm),
- `torch.optim.Adam`
- `torch.optim.Adadelta`
- `torch.optim.Adagrad`
- `torch.optim.RMSprop`
- `torch.optim.LBFGS`
- ...

So to use Adam with its default setting instead of vanilla SGD, we just have to change

```
optimizer = optim.SGD(model.parameters(), lr = eta)
```

into

```
optimizer = optim.Adam(model.parameters(), lr = eta)
```

⚠ The learning rate may have to be different if the functional was not properly scaled.